

"A wise developer always reads any requirements at least 11 times!"

Ahmad Yazdankhah

1 Objective

This document describes the structure of the Unary Calculator (UC) version 3.0 in details. It can be used by developers who intend to enhance the current features or add new features.

2 What is Unary Calculator (UC)?

Unary Calculator (UC) is a **programmable calculator** that works only with Unary numbers.

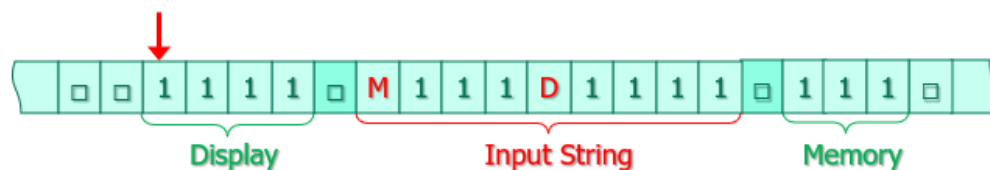
UC v2.0, with 9 instructions, was implemented in the previous semesters and is given to you. In this semester, you are going to add new instructions as will be described in the next sections.

UC has a **Memory** and a **display** whose sizes are unbounded. Therefore, it can store and display Unary numbers of any sizes.

These two components are simulated on the TM's tape as the following example shows.

Example

The following figure shows a TM's tape when it halted after executing the input string (will be covered shortly):



Note the 2 blanks that act as separators between Display, Input String, and Memory.

2.1 UC's Instructions Set: A Quick Overview

The following table summarizes UC's instructions. The next sections cover the details of each instruction.

	Mnemonic	Symbolic	Description
Already Implemented in V2.0	Mn	$n \rightarrow M$	Load the Unary number n into Memory
	MD	$D \rightarrow M$	Replicate the contents of Display into Memory
	MC	$\square \rightarrow M$	Clear (= blank) the contents of Memory
	Dn	$n \rightarrow D$	Load the Unary number n into Display
	DM	$M \rightarrow D$	Replicate the contents of Memory into Display
	DC	$\square \rightarrow D$	Clear (= blank) the contents of Display
	An	$D + n \rightarrow D$	Add the Unary number n to the contents of Display and store the result in Display
	AM	$D + M \rightarrow D$	Add the contents of Memory to the contents of Display and store the result in Display
	Sn	$D - n \rightarrow D$	Subtract the Unary number n from the contents of Display and store the result in Display If $D \leq n$, $\square \rightarrow D$
Enhancements	SM	$D - M \rightarrow D$	Subtract the contents of Memory from the contents of Display and store the result in Display
	Tn	$D * n \rightarrow D$	Multiply (Times) the Unary number n by the contents of Display and store the result in Display
	TM	$D * M \rightarrow D$	Multiply (Times) the contents of Memory by the contents of Display and store the result in Display

Notes

1. We used "**M**" for Memory, "**n**" for a Unary number, "**D**" for Display, " $\square$ " for blank (= Clear), "**A**" for Add, "**S**" for Subtract, and "**T**" for multiplication (Times).
2. The result of an operation such as "Add", "Subtract", and "Multiply" always goes to **Display**. In computer science, this is the concept of "**accumulator**". In other words, **Display** in UC acts as an accumulator.
3. Each mnemonic has 2 parts like XY. X is the destination (such as **Memory** or **Display**) or the operation (such as Add, Subtract, or Multiplication) and Y is the source, such as n as a Unary number, or C for clear (= blank), or M as the Memory, or D as Display.

2.2 Implemented Instructions Details

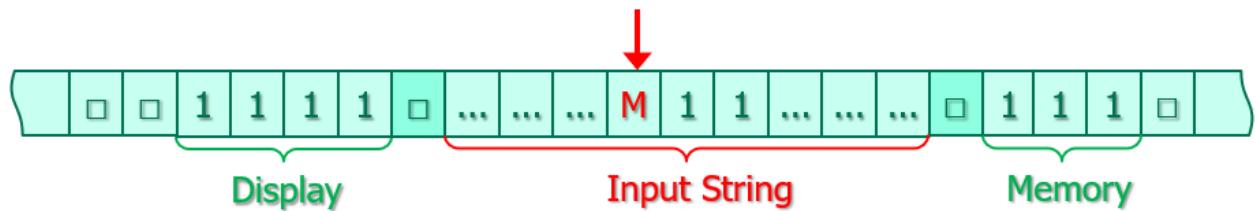
2.2.1 Mn ($n \rightarrow M$)

This instruction loads the **unary number n** into **Memory**, **overwriting** any value previously stored in **Memory**.

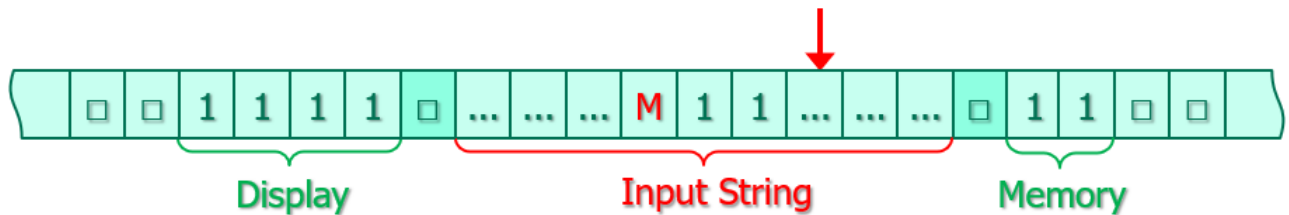
It does **not change** the contents of **Display**.

Example

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing **M11** are:



After the execution of **M11**, the contents of Display and Memory are:



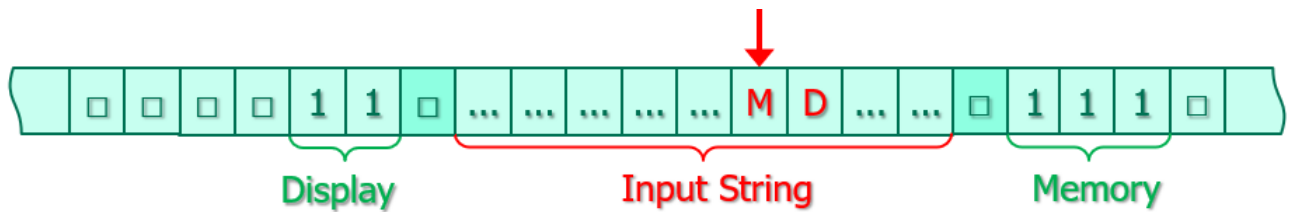
2.2.2 MD ($D \rightarrow M$)

This instruction **copies** the contents of **Display** into **Memory**, **overwriting** any value previously stored in **Memory**.

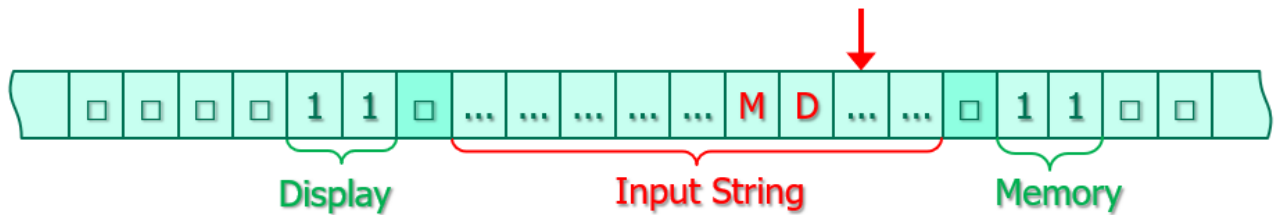
It is a **copy** operation (not a move), so the contents of **Display** remain **unchanged**.

Example

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing the **MD** are:



After the execution of **MD**, the contents of Display and Memory are:



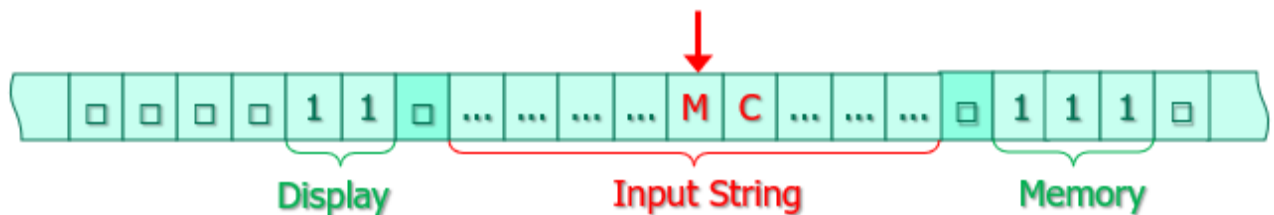
2.2.3 MC ($\square \rightarrow M$)

This instruction **clears** (= blanks) the contents of **Memory**.

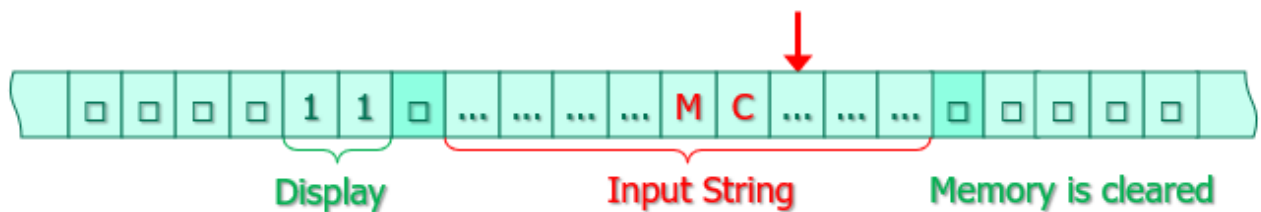
It does **not change** the contents of **Display**.

Example

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing the **MC** are:



After the execution of **MC**, the contents of Display and Memory are:



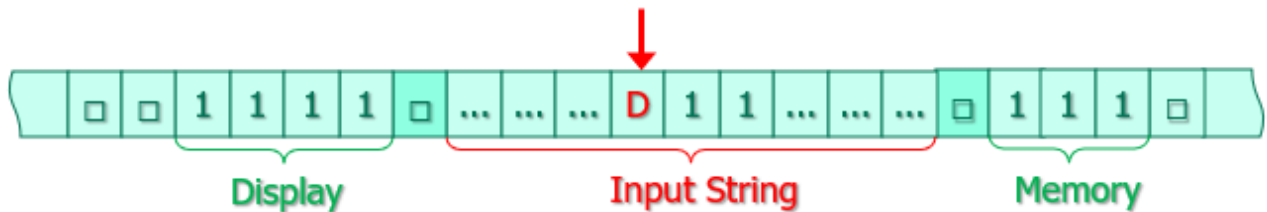
2.2.4 Dn ($n \rightarrow D$)

This instruction **loads** the Unary number **n** into **Display**, **overwriting** any value previously stored in **Display**.

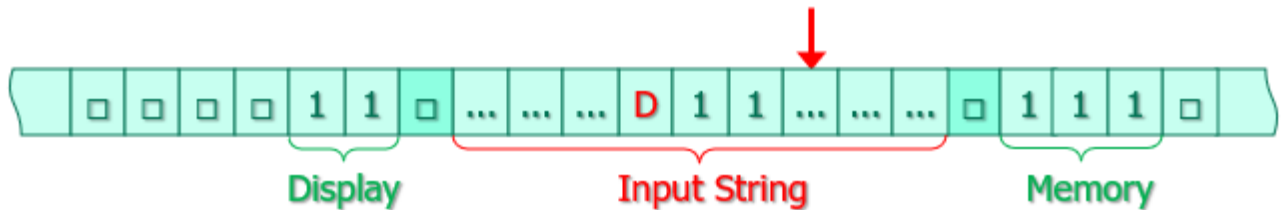
It does **not change** the contents of **Memory**.

Example

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing **D11** are:



After the execution of **D11**, the contents of Display and Memory are:



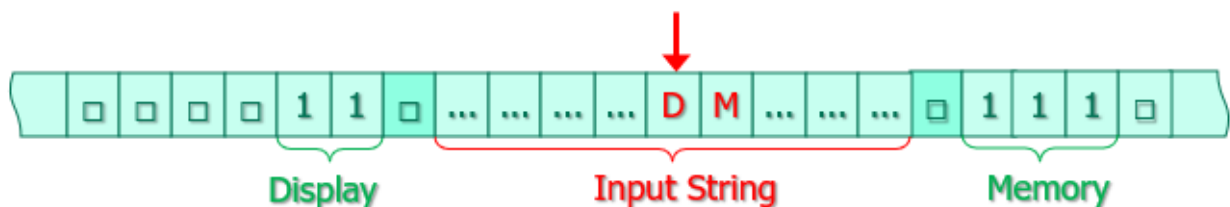
2.2.5 DM ($M \rightarrow D$)

This instruction **copies** the contents of **Memory** into **Display**, **overwriting** any value previously stored in **Display**.

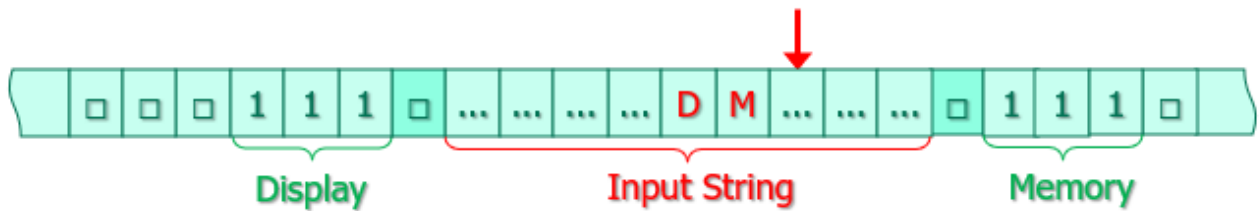
It does **not change** the contents of **Memory**.

Example

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing the **DM** are:



After the execution of **DM**, the contents of Display and Memory are:



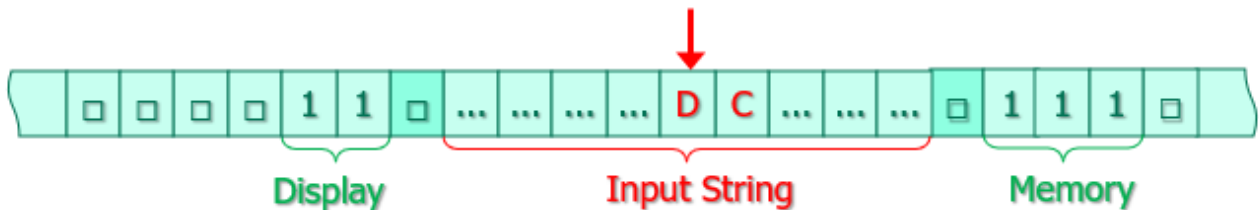
2.2.6 DC ($\square \rightarrow D$)

This instruction **Clears** (= blanks) the contents of **Display**.

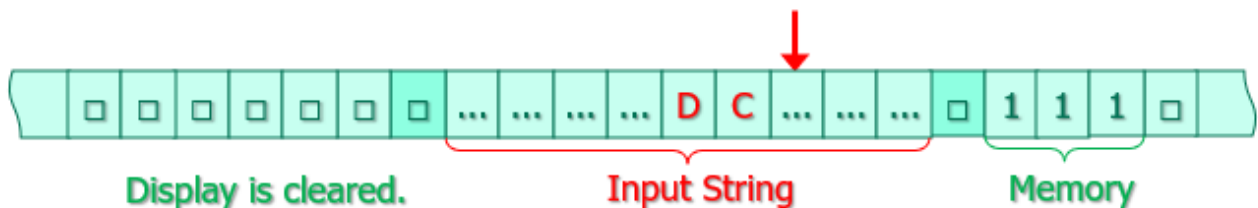
It does **not change** the contents of **Memory**.

Example

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing the **DC** are:



After the execution of **DC**, the contents of Display and Memory are:



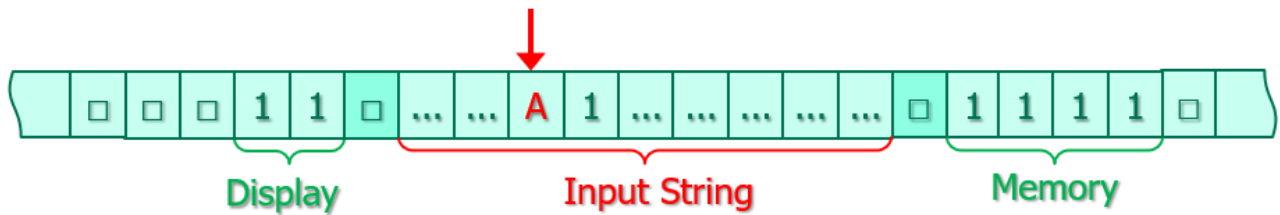
2.2.7 An ($D + n \rightarrow D$)

This instruction **adds** the Unary number **n** to the contents of **Display** and stores the result in **Display**.

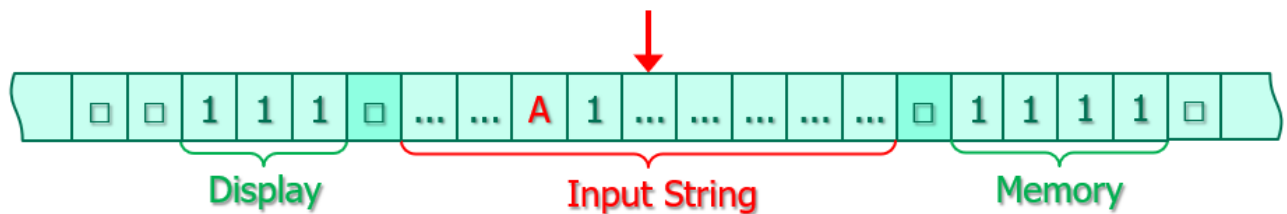
It does **not change** the contents of **Memory**.

Example

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing the **A1** are:



After the execution of **A1**, the contents of Display and Memory are:



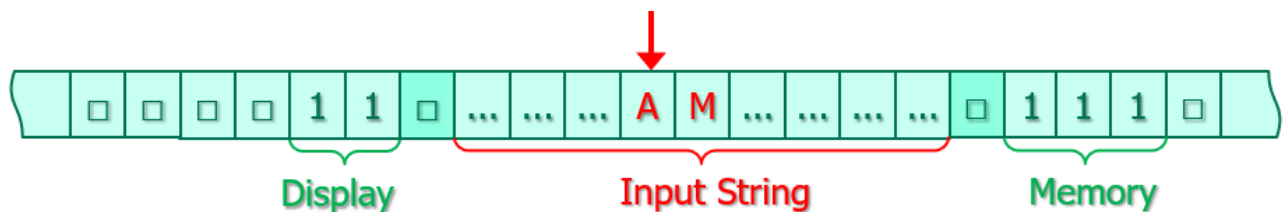
2.2.8 AM ($D + M \rightarrow D$)

This instruction **adds** the contents of **Memory** to the contents of **Display** and stores the result in **Display**.

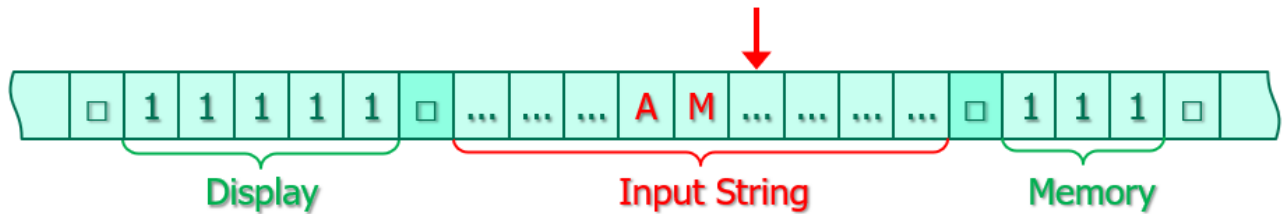
It does **not change** the contents of **Memory**.

Example

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing the **AM** are:



After the execution of **AM**, the contents of Display and Memory are:



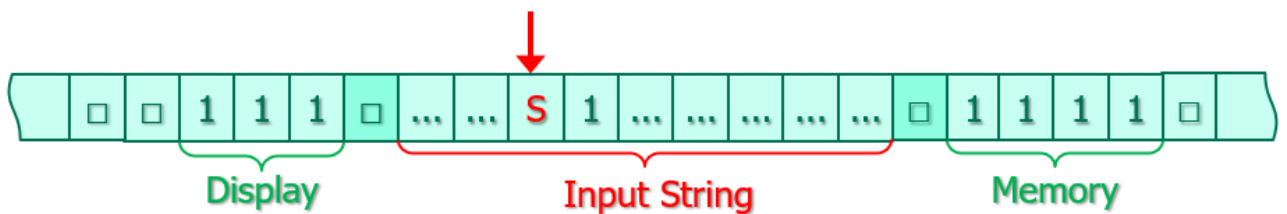
2.2.9 $S_n (D - n \rightarrow D)$

This instruction **subtracts** the Unary number **n** from the contents of **Display** and stores the result in **Display**.

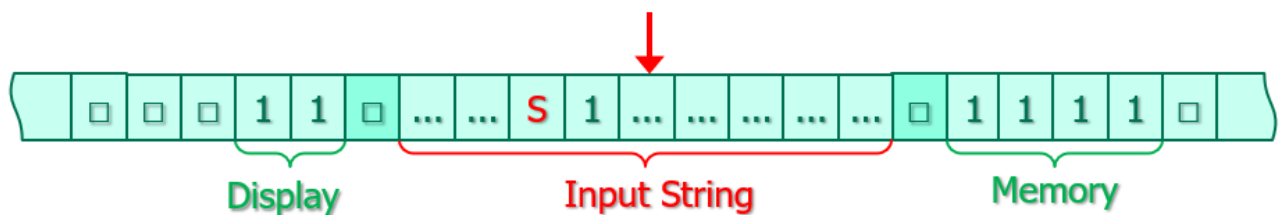
It does **not change** the contents of **Memory**.

Example #1

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing the **S1** are:



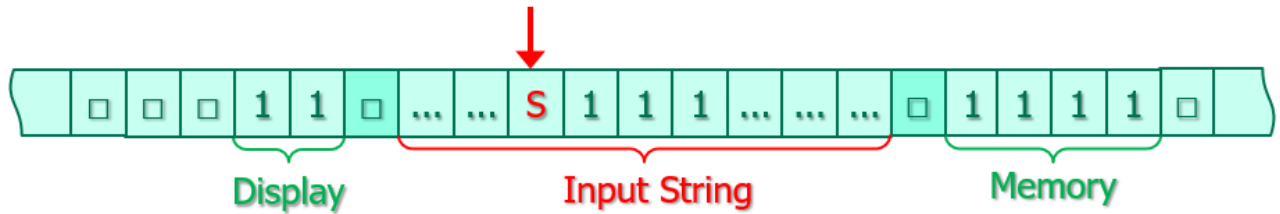
After the execution of **S1**, the contents of Display and Memory are:



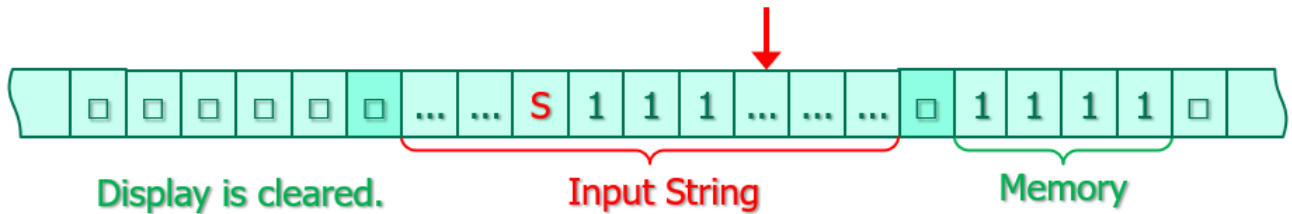
Note: We know that Unary numbers don't have **negative numbers**. Therefore, **if the result of a subtraction is negative, UC shows blank.**

Example #2

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing the **S111** are:



After the execution of **S111**, the contents of Display and Memory are:



2.3 Enhancement Instructions Details

The following instructions shall be implemented in this term's project.

2.3.1 SM (D - M → D)

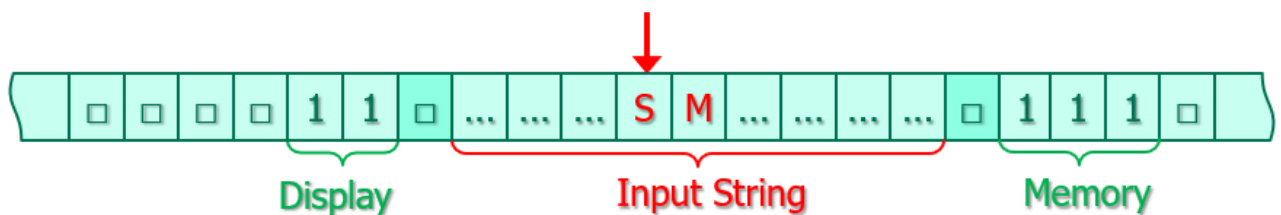
This instruction **subtracts** the contents of **Memory** from the contents of **Display** and stores the result in **Display**.

It does **not change** the contents of **Memory**.

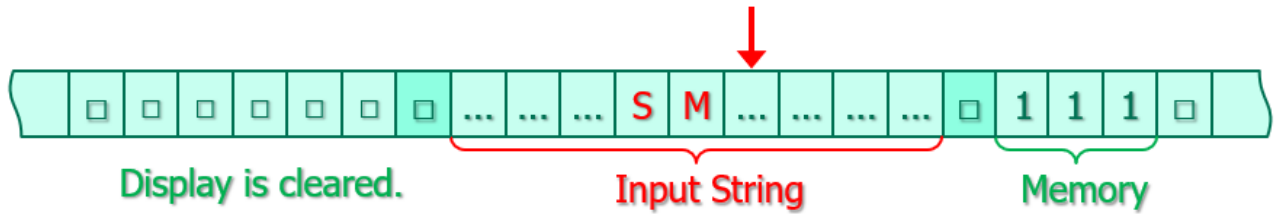
Note: We know that Unary numbers don't have **negative numbers**. Therefore, **if the result of a subtraction is negative, UC shows blank**.

Example #1

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing the **SM** are:

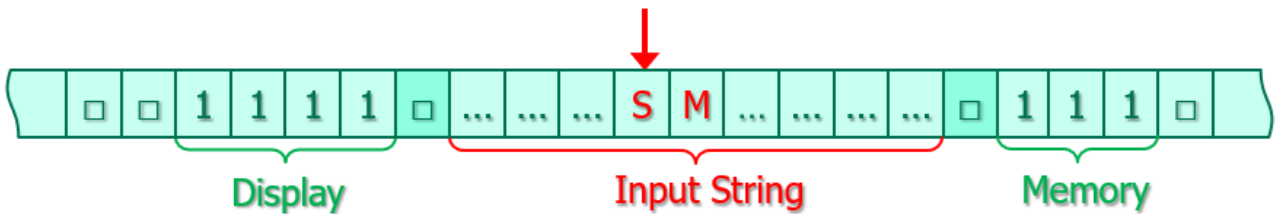


After the execution of **SM**, the contents of Display and Memory are:

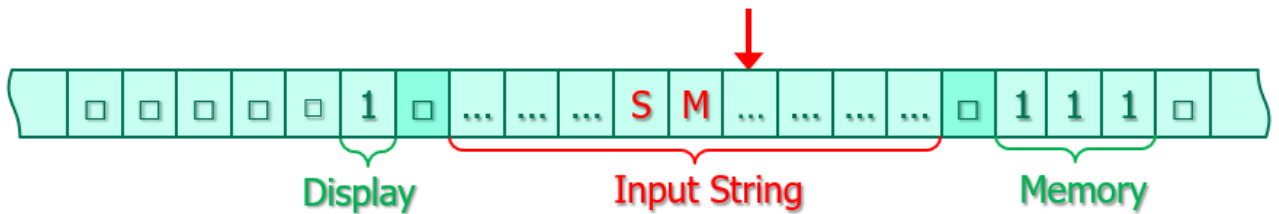


Example #2

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing the **SM** are:



After the execution of **SM**, the contents of Display and Memory are:



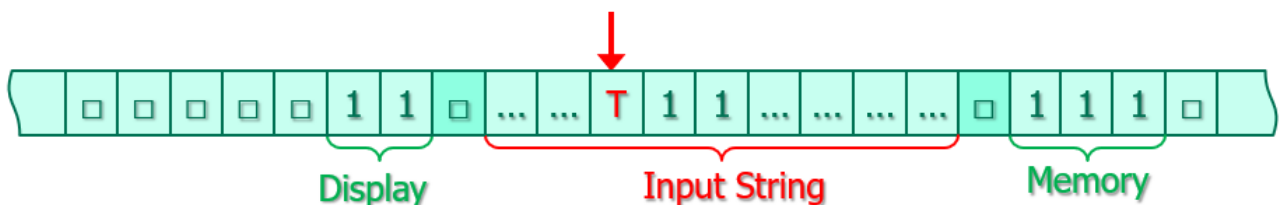
2.3.2 $T_n (D * n \rightarrow D)$

This instruction **multiplies** the Unary number **n** by the contents of **Display** and stores the result in **Display**.

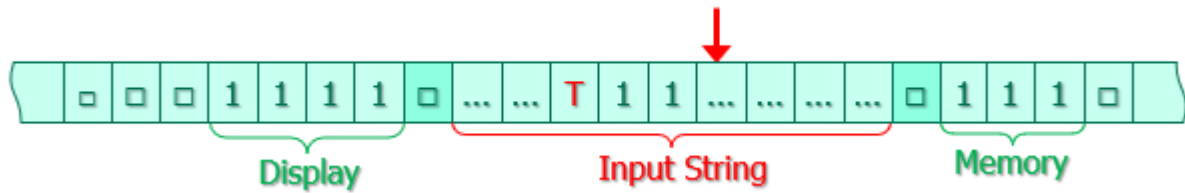
It does **not change** the contents of **Memory**.

Example

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing the **T11** are:



After the execution of **T11**, the contents of Display and Memory are:



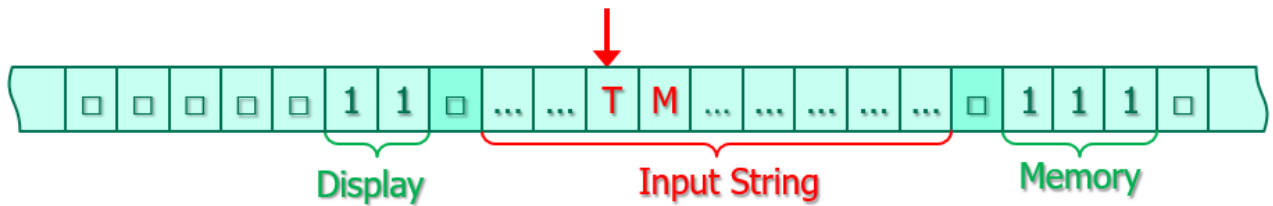
2.3.3 TM ($D * M \rightarrow D$)

This instruction **multiplies** the contents of **Memory** by the contents of **Display** and stores the result in **Display**.

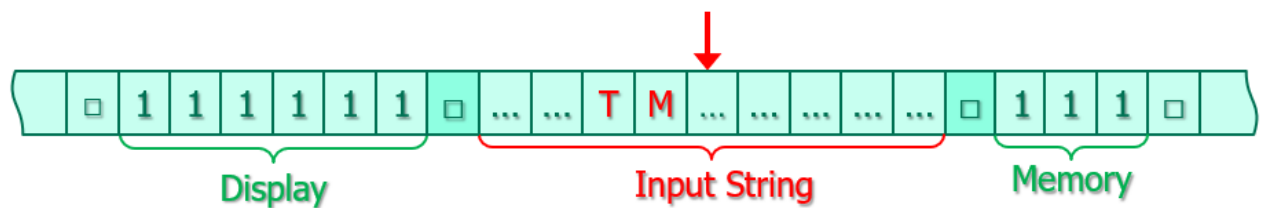
It does **not change** the contents of **Memory**.

Example

Assume we are in the middle of a computation and the contents of Display and Memory **before** executing the **TM** are:



After the execution of **TM**, the contents of Display and Memory are:



3 Examples

3.1 Valid Input Strings

The following table shows **a few examples** of input strings of the TM and the **expected Display content** when it halts.

In the following table, **Display** is equivalent to what is shown on the “**Output**” of the JFLAP and **Memory does not exist in JFLAP and showing it here is just for your information.**

Input String	Display	Memory	Result
M111		111	Accept
D111	111		Accept
DM			Accept
MD			Accept
MC			Accept
A11	11		Accept
D11A1	111		Accept
D11M111AM	11111	111	Accept
D111S1MD	11	11	Accept
MCD111S1MD	11	11	Accept
AMA11MDS11		11	Accept
D1111D111DCA111	111		Accept
S11AMD11A111S11MD	111	111	Accept
M111D1111SM	1	111	Accept
M11D111T11	111111	11	Accept
M111D11TM	111111	111	Accept

Notes

1. Each row of the above table is one test case.
2. As the above table shows, you can chain any number of instructions, and **the TM is going to execute them one by one from the left to the right** and then halts (no infinite loop).
3. **The final content of Memory is shown in the above table only for your information. Only the contents of Display must be shown by the TM when it halts.**

4. We know there is no **Zero** in the Unary numbers. Therefore, to show **Zero** in UC, we use **blank** on Display and Memory.
5. Recall that the TM must “Accept” the string (Result in the above table), otherwise, JFLAP does not show the Output (= Display of the TM).
6. These are just **a few examples** to show how the TM should work. **It's designer's responsibility to create tricky test cases and try to break their own design!**

3.2 Invalid Input Strings

For simplicity, we assume no invalid input strings are allowed. Therefore, **the TM does not need to detect or react against invalid input strings.**

Note that the input strings of the TM are solely made from the symbols of Σ . Therefore, any strings made from non- Σ are considered as invalid inputs.

The following table shows **a few examples** of invalid input strings.

Invalid Input Example	Reason
	Lambda is not allowed.
A12	$2 \notin \Sigma$
MD1 MC11	Wrong syntax. MD and MC don't have operand.
M D A S T	Wrong syntax. M, D, A, S, and T need to be followed with an operand (= Unary number) or another symbol.
MS	Wrong syntax. MS is not a valid instruction.
111A	Wrong syntax

4 Characteristics of UC's TM

UC has been simulated by a "**standard TM**" that has the following characteristics.

1. The **input string of the TM** is a sequence of instructions that shall be executed by the TM from the left to the right.
2. When the TM halts, the contents of the Display, that is the result of the final calculations, shall be shown on Display (= JFLAP's output).
3. $\Sigma = \{ M, D, C, A, S, T, 1 \}$
The input strings of the TM must be made from this alphabet. It means that we are not allowed to input a string made from symbols outside of this alphabet.
For example, K11 is not allowed to be an input string. Therefore, **TM does NOT need to detect any kind of errors.**
4. $\Gamma = \Sigma \cup \{ \square \} \cup \{ X, Y, Z \} \cup \{ x: x \text{ is any other symbols that you use in your design} \}$
The symbols you use in your design are transparent to the users of UC.
Note: X, Y, and Z are already used in the implementation of V2.0.